n = 83 K600 rank probe: the rank-cut loop in the target's Frobenius frame, end-to-end cold

2026-10-19

n = 83 K600 rank probe: the rank-cut loop in the target's Frobenius frame, end-to-end cold

Status (2026-10-19): protocol frozen; Tier 1 run pending. The protocol (frozen 2026-10-19T06:52:47Z) and the schedule were committed before any Tier 1 timing. `.github/workflows/n83-rankprobe.yml` runs them on one GitHub runner. Everything below that is not from an archived run comes from the cloud-agent VM. Those numbers are **local diagnostics**, not timing evidence (see `smoke/`).

Question

The n = 83 a = 1 K600 one-target index-calculus baseline is the rank-cut producer, `koblitz-n83-rankcut-20261016` (PR #970, run 38891404048). It was promoted end to end over the stacked producer at 1.3336 [1.3300, 1.3377]: end-to-end cold 195.7 s, S 7.52, gap to the tuned rho rT770 65.7 [55.4, 80.2]. Its rank stage is still 88.5% of end-to-end cold (173.2 of 195.7 s): 600 attempts, a mean of 1,790,030 probes per attempt, 161.1 ns per probe on the runner. Exactness fixes the attempt and probe counts, so the lever is still the wall time per probe.

The change: run the rank-cut loop in the target's Frobenius frame. This removes the two per-candidate normal-to-polynomial table maps and keeps every partner word, key, probe and relation. Does it lower the **end-to-end cold** cost on new targets and new rank seeds, with every rank relation, every per-attempt probe count, every pair order, every scalar and the factor-log table unchanged? This is an iteration under AGENTS.md "Success criterion". A promoted result becomes the new n = 83 row.

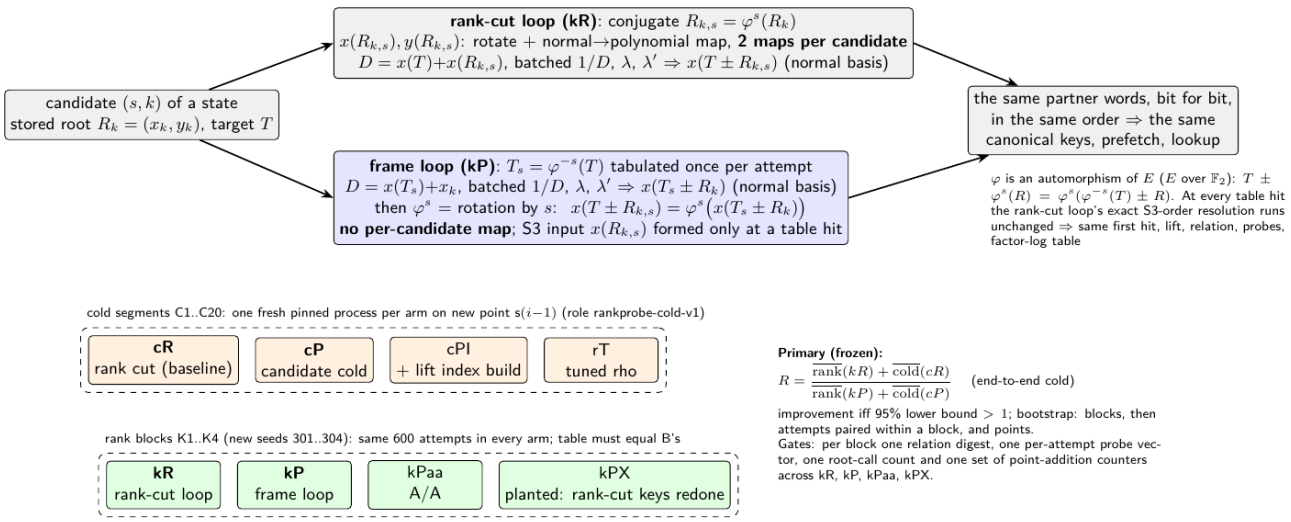


Figure 1: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

Where a rank-stage probe's time goes (local profile; diagnostic)

Source: `smoke/profile_local.json`, from the env-guarded `KIC_PROF3` block in `smoke/profile.patch` (a scratch copy of this producer, not a frozen one). The workload is the first rank target of seed 201, using the rank stage's configuration: compact table, folded index, lift_scan, Feistel order. The same 6,000 states are re-scanned with the stages added one at a time. The table gives the median over 15 interleaved rounds, in ns per probe (two probes per candidate). The VM is a shared 4-vCPU Xeon at 2.1 GHz with transparent huge pages set to `madvise`, where the runner uses `always`.

Stage of one probe	rank-cut loop (kR)	frame loop (kP)
rebuild the folded state	0.4	0.4
y (and x) of the two stored roots, once per state	3.7	3.5
per-candidate inputs: rotate + normal-to-polynomial map of x and y, prefix products	**25.3**	**9.3** (prefix products only)
batched inversion (one `invert128` per state, 3 multiplications per candidate)	20.7	22.0
lambda products, two normal-basis maps, both partners	39.1	36.5
lift_scan canonical keys of both partners	54.3	62.3
prefetch and key store	57.2	49.7
lookups and hit check	57.5	49.6
cumulative	**258.0**	**233.2**
the production loop on the same states	268.2	233.2

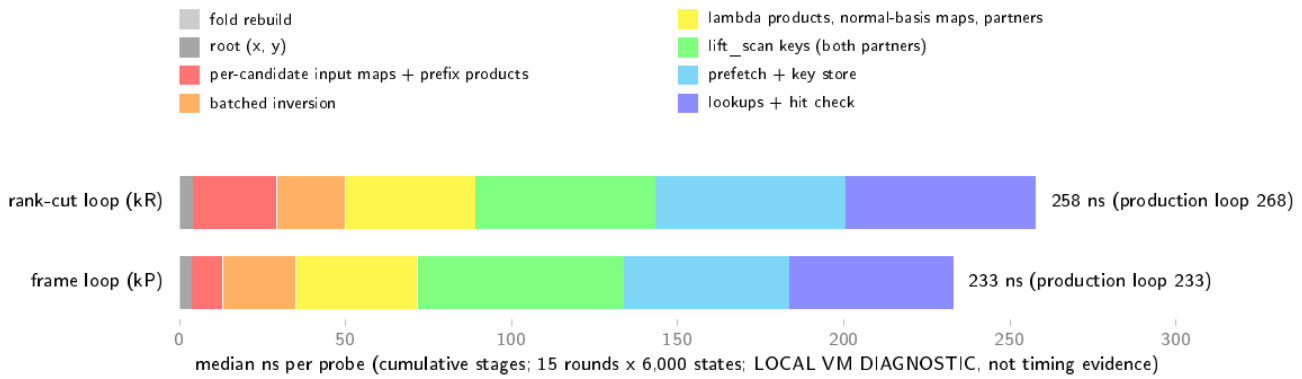


Figure 2: Local profile, median ns per probe per stage (diagnostic; source figures/profile.tex, rendered by `make_profile_figure.py` from `smoke/profile_local.json`).

The isolated costs of single operations (the median of 15 rounds, on 4,096 independent words) are:

- a normal-basis map (eleven byte-table lookups): 14.3–14.6 ns;
- a field multiplication (a call into the library's `pclmulqdq` fold): 16.9 ns;
- lift_scan: 40.8 ns;
- `invert128`: 433 ns.

Reading. The runner's 161 ns has the same structure with a different scale. Its lookups are about 7% of a probe there, because huge pages are always on. Its arithmetic is the rest.

Choice. The table splits the probe into pieces that cannot be removed and pieces that can:

- **lift_scan (about 54 ns).** This is the largest piece, but it is the key, so it cannot be skipped. A branch-free `u128` rewrite (checked equal on 65,536 words; `smoke/lift_scan_u128_micro.rs`) was only about 1.15x faster. The data-dependent candidate loop dominates it.
- **Lambda step (about 39 ns).** Its two maps produce the two partner words in the normal basis, which the keys need.
- **Lookups.** These are small on the runner.
- **Per-candidate input maps (about 25 ns).** This is the largest piece that can be removed entirely and exactly: it exists only to put a conjugate root into the polynomial basis.

The candidate ideas were judged against this profile:

- **Shared work between the two partners.** x(T+R) and x(T−R) already share the inversion (`lambda' = lambda + x/D`). The two partner words are unrelated, so their least rotations share no work.
- **A larger inversion batch.** One `invert128` per 166 candidates is about 1.3 ns per probe, so at most about 1 ns can be saved.
- **Working natively in the normal basis.** Multiplication in the normal basis costs more than the two maps it would save.
- **SIMD carry-less multiply.** A global `+pclmulqdq` build cut the isolated multiplication from 16.9 to 11.1 ns. Its production loops did not separate from the default build's on the VM (medians 287 against 268 ns for the rank-cut loop, inside the VM's round-to-round spread of 0.88–1.50x).

So the candidate is the frame change alone. cRI (lift_scan in the index build) runs as the separate, pre-registered secondary `cPI`.

The change: `KIC_RANK_SOLVE=frame`

`make_online_source.py` derives the producer in two steps:

1. `make_rankcut_source.py` is the rank-cut experiment's `make_online_source.py`, verbatim, with its four verbatim parents. Its output must hash to the rank-cut protocol's frozen producer digest, `c390733e...`. The derivation checks this.
2. Nine counted, exact replacements add two values to `KIC_RANK_SOLVE`, which only the rank stage reads (`padd` and `s3` are unchanged):
   - `frame` replaces the rank-cut loop (`scan_state128_padd`) with `scan_state128_frame`. The rank-cut loop forms, for each shift s and stored root R_k, the conjugate R_{k,s} = φ^s(R_k). To do that it rotates x and y in the normal basis and maps both to the polynomial basis, two table maps per candidate, before adding R_{k,s} to T. The frame loop instead tabulates T_s = φ^(−s)(T) once per attempt: x(T_s) and y(T_s) in the polynomial basis, and x(T_s) + a in the normal basis. It then uses the stored root's own (x_k, y_k):
     - `D = x(T_s) + x_k`, with one batched inversion;
     - `lambda = (y(T_s) + y_k) / D`;
     - `lambda' = lambda + x_k / D`;
     - `x(T_s+-R_k)=lambda^2+lambda+x(T_s)+x_k+a`, in the normal basis.

     It then applies φ^s, which is a rotation of the normal-basis word. The S3 input x(R_{k,s}) is formed only at a root-table hit or at an exceptional input. That leaves five field multiplications and two table maps per candidate, against the rank-cut loop's five and four.
   - `frame_plant` is the planted positive control. It is `frame`, plus the rank-cut loop's whole key derivation redone on every candidate of every state: the per-candidate maps, its own batched inversion, its partner formulas and the canonical keys. The result is compared with the emitted keys field by field (a panic on any difference), then discarded.

Why the output is exact.

- E is defined over F_2, so φ(x, y) = (x^2, y^2) is a group automorphism of E(F_{2^83}). Therefore T ± φ^s(R) = $\varphi^{s(\varphi}$(−s)(T) ± R). The two loops compute the same field elements, so the emitted normal-basis partner words are equal bit for bit.

- They are emitted in the same order: minus then plus, with the same y recovered by the half-trace. So the canonical keys, the partner shifts, the candidate indices and the probe positions are the rank-cut loop's.
- The exceptional inputs coincide:
  - x(R_{k,s}) = x(T) iff x_k = x(T_s);
  - x(R_{k,s}) = 0 iff x_k = 0;
  - x(T) = 0 iff x(T_s) = 0.

  These inputs are solved by the S3 solver on the rank-cut loop's S3 input. A root that fails the curve check falls back to the compact loop, exactly as in the rank-cut loop.
- At every root-table hit the rank-cut loop's exact S3-order resolution runs unchanged on the same S3 input.
- So the first hit, the lift, the relation and the probe count of every attempt are the rank-cut loop's, and through it the S3 loop's. The echelon and the factor-log table are B's.
- The setup, the index, the online target scan and the `s3` and `padd` loops are the rank-cut producer's code, unchanged.

Local exactness and diagnostics (cloud-agent VM)

See `smoke/README.md`. In short:

- **Tests.** All 18 producer tests pass, including three new ones:
  - the Frobenius identity on random points;
  - the frame keys equal to the rank-cut keys on random states;
  - `PIPE_FRAME`, `PIPE_FRAME_PLANT` and `PIPE_FRAME_SWAPPED` each return the S3 and rank-cut loops' relation, intermediates, probe count, root calls and relation checks on 40 planted targets, in three index and order settings.

  The 4 tuned-rho tests and lift2's 24 tests also pass.
- **Full K600 rank stages.** These used rank seed 201 on point s000 of the rank-cut panel, the inputs of run 38891404048's block K1, with `padd`, `frame` and `frame_plant` run as three concurrent processes. All three reproduced the archived kR row exactly:
  - the relation digest;
  - all 600 per-attempt probe counts (1,080,101,628 probes);
  - the root-call total;
  - the counters: 600 resolved hits, 283 swapped pairs, 0 fallback states, 0 exceptional inputs;
  - B's factor-log table, log for log;
  - the target relation, scalar and probe count;
  - B's index digests.

  `frame_plant` compared all 540,100,256 candidates key by key. Each `frame` stage built exactly 600 frames, one per attempt. The local stage times were 279.6 s (`padd`), 258.6 s (`frame`) and 452.4 s (`frame_plant`), 1.08x for frame over padd. These are diagnostics only.
- **Runner, wrapper and verifier.** `run_schedule.py` ran O1 and C1, and every row was valid. Q0 matched the archived Feistel identity (1,987,852 probes) in cR and cP. s000 gave one relation, one probe count and one scalar across cR, cP, cPI and rT, and the oracle replay passed. The first two K1 rows (kR and kP, with the new seed 301) are both valid. They give one relation digest, one 600-entry probe vector, one root-call count and one set of counters, with B's table.
- **Verifier statistics on synthetic rows.** These are run 38891404048's rows, renamed. A planted 1.08x rank effect gives R = 1.0698 [1.0677, 1.0718], "improvement". No effect gives 0.9997 [0.9977, 1.0018], "not separated". The controls pass in both cases.

Frozen Tier 1 design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, and one fresh process per run. Every IC arm runs the rank-probe producer (`ic_online_rankprobe_r770`) with the 16-byte compact table over the folded index, lift_scan, the Feistel order and the prefetch pipeline.

Arm	Kind	Rank solve	Role
`cR`	cold	(none)	**baseline** cold: the rank-cut producer's cold path
`cP`	cold	(none)	**candidate** cold (the same cold code; its own processes)
`cPI`	cold	(none)	pre-registered secondary: cP with lift_scan in the index build (cRI of the rank-cut run)
`rT`	rho	—	tuned rho rT770 (Fold/64/DP6), seed 880000 + point seed
`kR`	rank	`padd`	**baseline** rank stage (the rank-cut loop)
`kP`	rank	`frame`	**candidate** rank stage
`kPaa`	rank	`frame`	A/A of kP
`kPX`	rank	`frame_plant`	planted positive control

- **Cold panel.** 20 new public points (`inputs/cold_targets.json`, role `rankprobe-cold-v1`, built by `make_targets.py`). They are checked disjoint from every earlier n = 83 panel of the law, all regenerated:
  - Q0 and B's panel;
  - scanorder-heldout, combined-primary-v1, probepipe-primary-v1, statefold-cold-v1, tablefilter-primary-v1 and stacked-cold-v1;
  - rankcut-cold-v1;
  - the 64 `eval` points of n131-ref-walk-20261015.

  No scalar is known for any of them.
- **Rank blocks.** `K1`–`K4` use the new rank seeds 301–304, so 2,400 new rank targets. Each block runs one process of each rank arm: setup, the rank stage to full rank, linear algebra, then one target (s000–s003).
- **Segment order.** O1 (cR and cP on Q0), then C1–C5, K1, C6–C10, K2, C11–C15, K3, C16–C20, K4. The order within each segment is shuffled with new seeds. The time guard is 300 minutes; the job limit is 350.

Primary (pre-registered): the end-to-end cold ratio.

- E2E(k, c) = the mean rank stage of rank arm k over the blocks, plus the mean cold time (setup + one-target online, cached table) of cold arm c over the 20 points.
- R = E2E(kR, cR) / E2E(kP, cP).
- Bootstrap, 10,000 draws, seed 20261019:
  - the blocks are resampled;
  - within each drawn block, the attempt indices are resampled and used for both kR and kP;
  - the points are resampled, paired across cR and cP.

  The gates make attempt i the same target with the same probes in both arms. Each process's non-attempt time is kept.
- **Improvement** iff the 95% lower bound of R exceeds 1; regression iff the upper bound is below 1; otherwise not separated. **Invalid** if any gate or control fails or any scheduled row is missing.

Controls.

- A/A: E2E(kP, cP) / E2E(kPaa, cR) and cold cP / cR must both be inside [0.97, 1.03].
- Planted: the per-attempt rank cost kPX / kP must have its lower bound above 1.

Gates.

- Every IC process uses the compact layout over the folded index, with B's index digests and 29,878,182 entries.
- Per point: one scalar across every IC process and rT (oracle replay), and one relation and one probe count across every IC process.
- Q0: the archived relation and scalar with 1,987,852 probes.
- Every rank process: full rank, the arm's `rank_solve` recorded, and a factor-log table equal to B's, log for log.

- **Per block, identical across kR, kP, kPaa and kPX:**
  - rank_relations_blake3;
  - the 600-entry per-attempt probe vector;
  - rank_root_calls_total;
  - the point-addition counters (fallback states, exceptional inputs, resolved hits, swapped pairs).
- Frame counters:
  - kR builds no frame;
  - kP, kPaa and kPX build exactly one frame per attempt;
  - only kPX checks inputs, and it must check at least one.

Secondaries. None of them decides.

- kR/kP per-attempt cost, overall and by tercile.
- ns per probe per arm, and kP/kPaa.
- Cold without rank, setup, online and peak RSS for cR/cP (expected 1).
- The pre-registered `cPI`: E2E(kR, cR) / E2E(kP, cPI) and cold cP / cPI.
- The counters.
- `S` per arm and the gaps to rT770.

Power (`power_check.py`, `analysis/power.json`). The prior is `inputs/power_prior.json`, built by `make_power_prior.py` from run 38891404048's kR/kRaa attempts (four blocks) and its cR/cS cold times (the same cold code).

- Power is 1.00 for 2, 3 and 4 blocks at end-to-end effects of 1.02, 1.03 (the minimal effect of interest), 1.05 and 1.10. This is from 200 simulations per cell with 300 bootstrap draws each, using the frozen paired block bootstrap.
- The rejection rate at no effect is 0.125, 0.080 and 0.085 for 2, 3 and 4 blocks. That is above the nominal 0.025. The prior's per-process A/A ratios (0.9969–1.0031) are a drift of about ±0.3% per process, which four resampled blocks resolve only partly. The protocol records this. An effect of 1.02 or more is detected with power 1.00, and the A/A control checks the drift on the run itself.
- The design runs 4 blocks, the frozen minimum.
- Expected effect (not an input): about 1.06–1.10 end to end if the local per-probe ratio (median 1.10) holds on the runner.

Runtime estimate. About 1.9 h including the build (up to about 2.5 h). A rank block takes about 950 s and a cold segment about 70 s.

Result

Pending: Tier 1 run on `claude/n83-rankprobe`.

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`make_online_source.py`, `make_rankcut_source.py`, `make_stacked_source.py`, `make_tablefilter_source.py`, `make_probepipe_source.py`, `make_combined_source.py`, `online/`, `lift2/`	the rank-probe producer (the rank-cut derivation verbatim, digest-checked, plus `KIC_RANK_SOLVE=frame|frame_plant`); tuned rho; the two-limb canonicalizer crate
`restore.py`, `check_log_table.py`	restore and digest-check crypto 770f6a69 and every local source; re-verify B's factor-log table
`make_targets.py`, `inputs/cold_targets.json`	the new cold panel
`inputs/archived_identities.json`	archived Q0 identity
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior (run 38891404048), simulation, result
`arm.py`, `run_schedule.py`	one arm per pinned process; the schedule runner

File	Role
`verify_receipt.py`	gates, statistics, decision rule; `result.json`, `result_table.md`
`figures/design.*`, `figures/profile.*`, `make_profile_figure.py`	design diagram; local profile graph (TikZ source, PDF, PNG)
`make_figures.py`	result figure, once the run is archived
`smoke/`	local diagnostics: profile and patch, exactness of full local rank stages, tests, runner smoke, verifier check
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`